

## IMPLEMENTATION REPORT

# PDF ingestion: strategy and logic

How four policy PDFs become searchable vectors in PostgreSQL, using a free local embedding model, and how each design choice was verified.

IMPLEMENTED / LIVE-VERIFIED / DEMO SCOPE

**PROJECT**

AI-assisted SupportOps

**REPORT DATE**

October 2, 2026

**SCOPE**

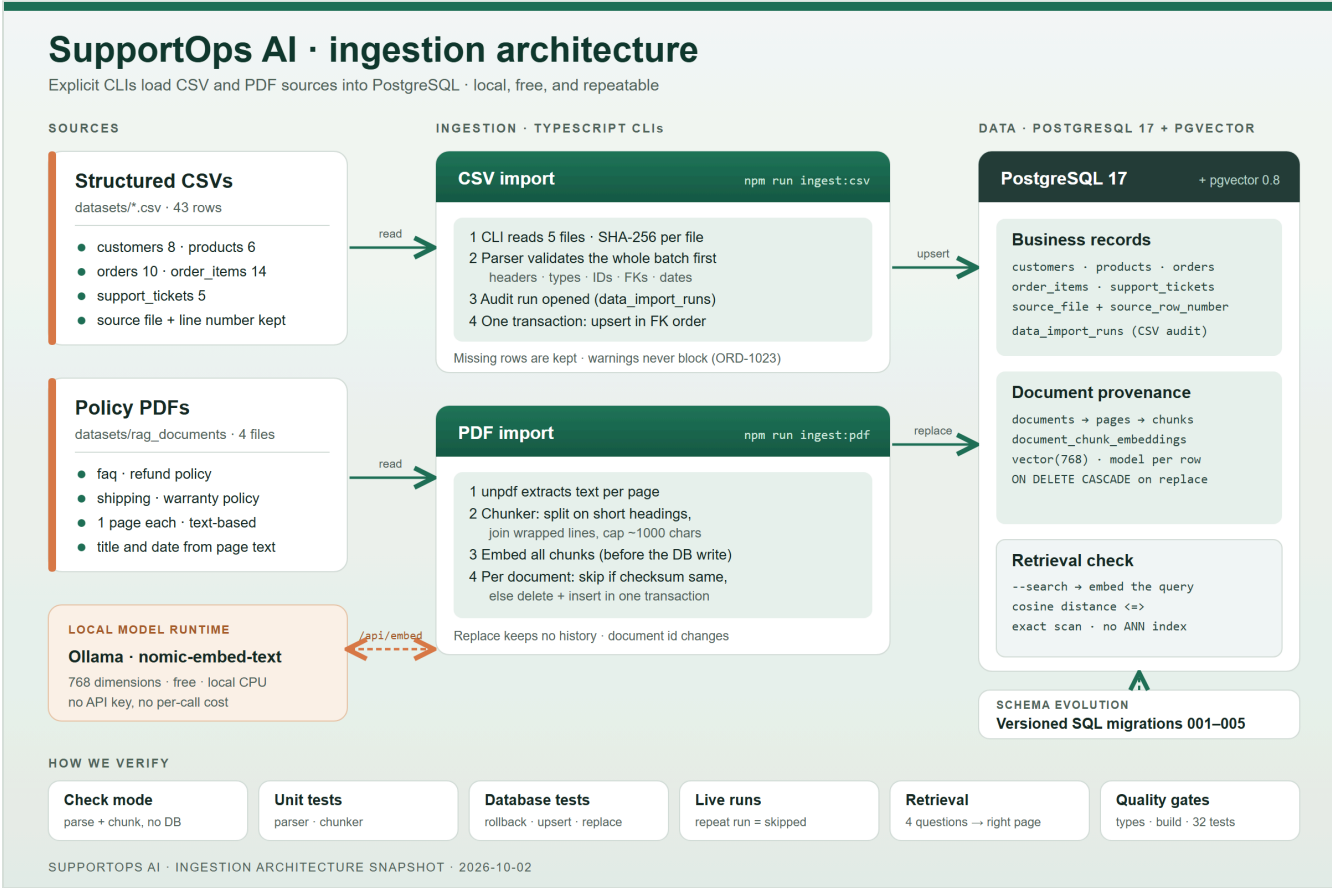
PDF extraction, chunking, embeddings, retrieval check

# A learning build that fits the missing pieces together

The goal was not a production document platform. It was to learn how the missing pieces fit: text extraction, chunking, embeddings, pgvector storage, and retrieval, with a demo that costs nothing to run.

| IN SCOPE                                            | DELIBERATELY OUT OF SCOPE                            |
|-----------------------------------------------------|------------------------------------------------------|
| The four PDFs in datasets/rag_documents/            | Upload endpoints, admin UI, background queues        |
| Per-page text extraction and heading-based chunking | OCR for scanned documents                            |
| Local embeddings through Ollama (nomic-embed-text)  | Hosted embedding APIs and any paid service           |
| pgvector storage and an exact cosine search         | ANN index, hybrid keyword search, reranking          |
| Skip unchanged files, replace changed ones          | Version history, a PDF audit table, chat integration |

# The ingestion process at a glance



Sources enter through explicit CLIs, are validated and transformed, and land in PostgreSQL. PDFs add a local Ollama call for embeddings. The row of checks along the bottom shows how each step is verified.

Orange marks sources and the local model. Green marks the application code. The dark card is PostgreSQL 17 with the pgvector extension.

## What we chose, and why

| DECISION             | CHOICE                                                                    | REASON AND TRADE-OFF                                                                                                                                                                    |
|----------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Execution model      | Explicit CLI ( <code>npm run ingest:pdf</code> )                          | Reproducible and easy to test, with no auth or UI surface. Heavy work stays out of request handlers. A real product would use an upload endpoint and a worker.                          |
| Layering             | CLI, import service, extractor, chunker, embedding client                 | Extractor and chunker are pure and database-free. The embedding client is replaceable by a fake in tests. The service owns the transaction and can be wrapped by an API later.          |
| Text extraction      | unpdf, pinned to 1.7.0                                                    | Small and Node-friendly. Later versions require Node 22, which conflicts with the server's stated Node 20 floor, so the version is pinned.                                              |
| Chunking             | Heading-based, about 1000 characters maximum                              | The policies have clear short headings, so a chunk is a coherent policy section that can be cited. A heuristic like this would not generalize to arbitrary PDFs.                        |
| Embedding model      | Ollama <code>nomic-embed-text</code> (768 dimensions)                     | Free, local, and small enough for CPU. Costs: running an extra service and a large Docker image.                                                                                        |
| Vector storage       | Separate <code>document_chunk_embeddings</code> table, key (chunk, model) | Chunk text stays the source of truth. Another model can be stored side by side. The column is fixed at <code>vector(768)</code> , so a different dimension needs a new column or table. |
| Search               | Exact cosine distance ( <code>&lt;=&gt;</code> ), no index                | With 12 chunks, a full scan is instant and has perfect recall. An HNSW index would only add tuning for no benefit at this size.                                                         |
| Change handling      | SHA-256 checksum: skip if unchanged, else replace                         | Re-runs are safe and cheap. Replacement is simple but loses history (see the open questions).                                                                                           |
| Transaction boundary | Embed first, then one transaction per document                            | No network call while a transaction is open. A failure rolls back that document only. This differs from the CSV import, which is all-or-nothing.                                        |
| Database image       | <code>pgvector/pgvector:pg17</code> plus migration 005                    | The official Postgres image lacks pgvector. Same major version, so the existing data volume is reused.                                                                                  |

## From a PDF file to a ranked answer

1. **Read and fingerprint.** Every `.pdf` in the folder is read and hashed with SHA-256. The relative path becomes the document's unique `source`. The document type comes from the filename.
2. **Extract.** `unpdf` returns text per page. The first line is the title, and a line containing "Last updated YYYY-MM-DD" provides the document date. Both come from the visible text, not the PDF metadata, which is anonymous.
3. **Chunk.** A line is a heading when it is at most 60 characters, has no trailing punctuation, and the next line does not start in lowercase. The lowercase test stops wrapped sentence fragments from being mistaken for headings. Body lines are joined, and each chunk is prefixed with "Title - Heading" so it carries its own context. Sections over about 1000 characters are split at sentence boundaries.
4. **Decide.** If the same source already exists with the same checksum and has embeddings for the model, the file is skipped. If the checksum changed, the document is replaced.
5. **Embed.** Chunk texts are sent in one request to Ollama's embed endpoint. `nomic-embed-text` expects a task prefix, so documents are sent with `search_document:` and queries with `search_query:`. The client rejects any response whose count or dimension (768) is wrong.
6. **Write.** One transaction deletes the old document if replacing (pages, chunks and embeddings cascade), then inserts the document, its pages, its chunks and their embeddings. Any error rolls the document back.
7. **Search.** A question is embedded as a query and compared to every stored vector for the same model with cosine distance. The nearest chunks return with filename, page number, heading and text. Lower distance means closer meaning.

**Why the same model must embed documents and queries:** vectors from different models are not comparable. The model name is stored on every embedding so a later switch is a re-embedding job, not a redesign.

## Options considered, and why a free local model

| OPTION                                                     | WHERE         | NOTES                                                                                                                                                                      |
|------------------------------------------------------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>nomic-embed-text (768)</code>                        | Local, Ollama | <b>Chosen.</b> About 274 MB, runs on CPU, free, and has asymmetric task prefixes for documents and queries. 768 is well under pgvector's 2000-dimension limit for indexes. |
| <code>all-minilm (384)</code>                              | Local, Ollama | Smaller and faster, with lower quality. A good fallback on weak hardware.                                                                                                  |
| <code>mxbai-embed-large (1024)</code>                      | Local, Ollama | Heavier, usually better quality. Reasonable if retrieval quality becomes the bottleneck.                                                                                   |
| <code>bge-m3 (1024)</code>                                 | Local, Ollama | Multilingual. Relevant if the documents were not English.                                                                                                                  |
| <b>OpenAI</b> <code>text-embedding-3-small / -large</code> | Hosted API    | Strong quality, simple to call, 1536 and 3072 dimensions (shortenable). Costs money, needs a key, and sends text out of your environment.                                  |
| <b>Cohere, Voyage, Google</b>                              | Hosted API    | Comparable trade-offs. Providers deprecate models, which forces re-embedding.                                                                                              |

Model names, dimensions and prices change often; check current provider documentation before relying on this table.

### Why we chose the free local option

- **Cost:** the demo must be free, and a hosted API needs a paid key.
- **Learning goal:** running the model locally shows every piece, with no black box between the application and the vectors.
- **Privacy:** nothing leaves the machine, which is a good default even for demo data.
- **Fit:** four short documents do not need a large model, and chunk quality matters more than model size at this scale.

**Practical costs we hit:** the `ollama/ollama` image is about 9 GB on disk, and the first pull from Docker Hub was slow and failed once on a network handshake. The embedding model itself is only 274 MB. A chat model is a separate, later choice with no schema impact.

# Replacement strategy and what is unresolved

| Replace or keep versions?                              |                                                                                                                                                  |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TODAY</b>                                           | A changed file deletes the document row and recreates it. The document id changes and no history is kept.                                        |
| <b>RISK</b>                                            | Citations or references to an old chunk id go stale. There is no way to answer what a policy said last month.                                    |
| <b>REAL PRODUCT</b>                                    | A stable document key plus version rows with effective dates. Old chunks stay but are marked inactive.                                           |
| Re-embed everything or only what changed?              |                                                                                                                                                  |
| <b>TODAY</b>                                           | The whole document is re-embedded on any change.                                                                                                 |
| <b>REAL PRODUCT</b>                                    | Hash each chunk and re-embed only changed chunks, to save time and cost on large documents.                                                      |
| What happens to a PDF that disappears from the folder? |                                                                                                                                                  |
| <b>TODAY</b>                                           | Nothing. Its rows remain, just as omitted CSV rows do.                                                                                           |
| <b>REAL PRODUCT</b>                                    | An explicit delete or archive action, with an audit trail.                                                                                       |
| Should a batch be all-or-nothing?                      |                                                                                                                                                  |
| <b>TODAY</b>                                           | Atomic per document. One failing file does not undo documents already written, unlike the CSV import.                                            |
| <b>REAL PRODUCT</b>                                    | Depends on whether documents are independent. Independent documents usually justify per-document commits with a clear report.                    |
| Model changes and concurrency                          |                                                                                                                                                  |
| <b>TODAY</b>                                           | The (chunk, model) key allows side-by-side embeddings, but there is no active-model flag or cleanup. Two runs at once are not guarded by a lock. |
| <b>REAL PRODUCT</b>                                    | An active-model setting, a re-embedding job, and a lock or job queue to prevent overlapping runs.                                                |
| Auditing                                               |                                                                                                                                                  |
| <b>TODAY</b>                                           | No PDF run table. Results are printed by the CLI. The CSV import has a persistent audit table.                                                   |
| <b>REAL PRODUCT</b>                                    | A run table with status, checksums, counts and sanitized errors, like the CSV one.                                                               |

## Evidence at each layer

| LAYER                        | CHECK                                                                                                       | RESULT                                                                                                                                    |
|------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Chunking, no services        | <code>npm run ingest:pdf -- --check</code> on the real PDFs                                                 | 12 chunks, three per document, split at the real section headings.                                                                        |
| Chunker unit tests           | Heading detection, wrapped-sentence handling, title and date, oversized split, chunk numbering across pages | Pass. One test caught a real bug where a wrapped line was treated as a heading, which led to the lowercase rule.                          |
| Service tests, live database | Fake embedder with deterministic vectors                                                                    | Pass: import, skip unchanged, replace without orphan chunks or embeddings, rollback when an embedding is missing, nearest-first ordering. |
| Embedding endpoint           | One real request to local Ollama                                                                            | Returned a 768-dimension vector, matching the column.                                                                                     |
| Live ingestion               | Run the import twice                                                                                        | First run imported 4 documents. The second run skipped all 4.                                                                             |
| Stored data                  | Row counts in PostgreSQL                                                                                    | 4 documents, 4 pages, 12 chunks, 12 embeddings. The existing 8 customers were intact.                                                     |
| Retrieval quality            | Four natural-language questions through the search command                                                  | See the table below.                                                                                                                      |
| Whole project                | Typecheck, production build, full test suite                                                                | All pass; 32 tests across 3 files.                                                                                                        |

| QUESTION                             | TOP MATCH                                     | DISTANCE |
|--------------------------------------|-----------------------------------------------|----------|
| How long do refunds take?            | Refund policy, 1. Standard Refund Eligibility | 0.177    |
| Is my laptop covered if I drop it?   | Warranty policy, 2. Exclusions                | 0.385    |
| When do refunds need human approval? | Refund policy, 3. Approval Rules              | 0.176    |
| How can I track my package?          | Shipping policy, 3. Tracking                  | 0.336    |

**An honest reading of the first result:** the match returns the 30-day refund window. The PDFs do not state how long a refund takes to process, so a human reviewer would call that match relevant but not a complete answer. Retrieval finds the right section; answering well is the job of a future chat model.

All tests and CLI runs used a temporary Node 22 runtime because the installed system Node is 18.16.0, below the Vite and Vitest requirement. The project's engine settings were not changed. Four sample questions are a smoke test, not an evaluation set.

### Known limits and next steps

- Add a PDF run-audit table and an advisory lock.
- Add an ANN index (HNSW) only if the chunk count grows to thousands.
- Add keyword search alongside vectors if exact terms are missed.
- Connect retrieval to the chat endpoint, then choose a chat model.
- Postgres reports a collation-version warning because the pgvector image uses a different glibc. It is harmless here; `ALTER DATABASE ... REFRESH COLLATION VERSION` clears it.